MASTER IN ASTROPHYSICS AND SPACE SCIENCE

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Optimisation & Monte Carlo Sampling

Introduction to Numerical Methods for Astrophysics

Fritz Ali Agildere

November 27, 2025

Overview

Overview

Frequentist View

- *With Prior Knowledge:* out of all n different but equally likely possible outcomes of an event, there are k that have a property A which is therefore measured with probability

$$P(A) = \frac{k}{n}$$

- *Without Prior Knowledge:* out of n independent measurements the outcome A occurs k times, defining

$$P(A) = \lim_{n \to \infty} \frac{k}{n}$$

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B
- *Kolmogorov Axioms* underpin the rules for calculating with probabilities
- $P(A, B) = P(A)\, P(B|A)$ quantifies the probability of measuring A and B simultaneously, and from assuming $P(A, B) = P(B, A)$ we find

Bayes' Theorem

$$P(A|B) = \frac{P(B|A)\, P(A)}{P(B)}$$

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \le x)$
- *Probability Density Function:* $f(x) = \frac{d}{dx} F(x)$

$$P(a \le X \le b) = \int_a^b f(x)\, dx = F(b) - F(a)$$

- *Normalization:* $1 = \int_{-\infty}^{+\infty} f(x)\, dx \qquad \lim_{x \to -\infty} F(x) = 0 \qquad \lim_{x \to +\infty} F(x) = 1$

Expectation Values

$$E[h(x)] = \int_{-\infty}^{+\infty} h(x)\, f(x)\, dx$$

- *Mean* or first raw moment: $\mu = E[X] = \int_{-\infty}^{+\infty} x f(x)\, dx$
- *Variance* or second raw moment: $\sigma^2 = E\left[(X - E[X])^2\right] = \int_{-\infty}^{+\infty} \left(x - E[x]\right)^2 f(x)\, dx$

Overview

Drawing Numbers

$$x \sim U(a, b) \qquad\qquad y \sim N(\mu, \sigma)$$

- *Uniform Distribution:* $U(a, b) = f(x|a, b) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$
- *Gaussian Distribution:* $N(\mu, \sigma) = f(y|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{y-\mu}{\sigma}\right)^2\right)$

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \dots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
  - *Linear Congruential*
  - *Linear Feedback Shift*
  - *Permutated Congruential*
- *Arbitrary Distributions:* obtained from uniformly distributed numbers by various methods

Overview

Motivation

- *Model Optimisation:* finding the best possible model parameters
  - maximising objective function *(Fitting)*
  - minimising loss function *(Machine Learning)*
- *Example:* modified blackbody spectrum for galactic dust emission (M. Juvela, *A&A* 2023)

$$I_\nu(T,\beta) \equiv \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

Example

$$I_\nu(T, \beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*
  - independent measurements generated by $I_\nu(\theta)$ with normally distributed uncertainty
  - infinitesimally small error in ν so that we only need σ_ν in I_ν direction
  - suppose we have no prior information on the parameter distribution

Example

- *Likelihood:*

$$-\log L(I_\nu|\nu, \sigma_\nu, \theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

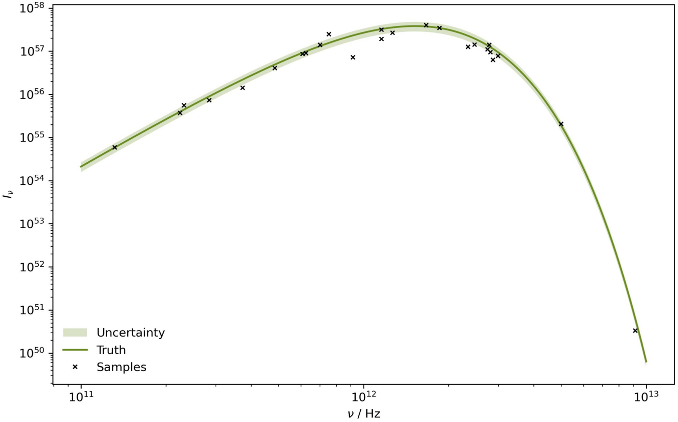
- *Posterior:*

$$\log P(\theta|I_\nu, \nu, \sigma_\nu) \propto -\chi^2(\theta)$$

- *Objective:* minimise the $\chi^2(\theta)$ measure to maximise the posterior probability

Measurements

How do we proceed from this dataset to maximise the likelihood for our nonlinear model?



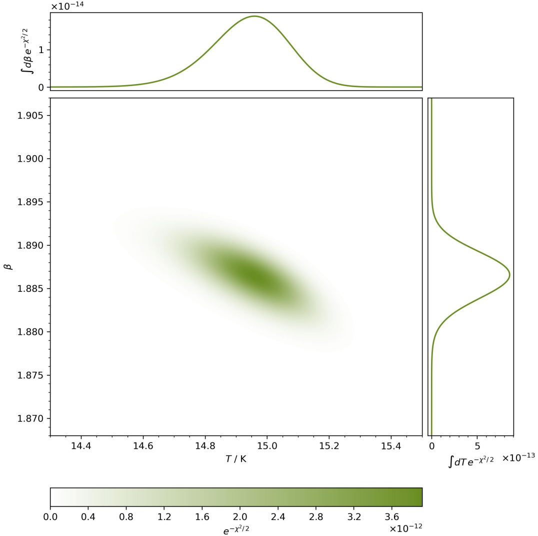
Overview

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter
  - from the literature we expect $(T, \beta) \in [10, 25] \times [1.5, 2.0]$ as constraints
  - resize these intervals by defining $T \mapsto \tilde{T} = T/30$ with $I_\nu(T, \beta) \to I_\nu(\tilde{T}, \beta)$
- *Problem:* as D increases we have N^D gridpoints, and with more complex parameter distributions, this procedure quickly becomes intractable due to computational limits

Parameter Grid

Can we think of more elegant ways to explore even very high dimensional parameter spaces?



General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature
  3. propose new step in parameter space
  4. decide whether proposal is a good step and update position accordingly
  5. repeat until some predefined cutoff is satisfied
- *Interactive Visualisations:* check out Ben Frederickson[1] and Andy Casey[2] to see some nice examples

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \dots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
  5. reflect, expand, contract or shrink based on results to move points downhill and repeat
- *Notes:*
  - extrapolates function to slowly move geometric blob through parameter space
  - zeroth order method since no derivatives are used

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*
  - traverses parameter space via random walk, can be very slow
  - zeroth order method since no derivatives are used
  - improve by considering last step and optimizing step range

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. initialize at some x_0
    2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
    3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
    - moves along direction of steepest descent at each point with fixed learning rate α
    - first order method since only the gradient is used
    - improve by performing line search for α at every step

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. start from some fixed large α_0 for each step k of the gradient descent
    2. set $\alpha_{j+1} = \tau\alpha_j$ until $f\left(x_k\right) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
    3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
    - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing k
    - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$
    - robust convergence guarantees, larger step size for flat areas avoids saddle points
    - first order method since only the gradient is used
    - typical settings are $\tau = \kappa = 0.5$ (L. Armijo, *Pac. J. Math.* 1966)

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - uses gradients from previous evaluations to estimate curvature
  - smooths oscillatory behaviour of naive gradient descent, especially in tight valleys with nearby gradients almost orthogonal to search direction
  - first order method since only the gradient is used

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha \tilde{m}_k \left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat
- *Notes:*
  - point behaves as iterative analogue to ball rolling in landscape with friction
  - hyperparameters $\beta_1, \beta_2 \in (0, 1)$ are forgetting factors for previous gradients, and $\varepsilon \ll 1$
  - first order method since only the gradient is used

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\dots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\dots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*
  - generalises Newton-Raphson method for finding roots of derivative to multiple dimensions
  - finds critical point, no matter if maximum, minimum or saddle point, sensitive to initialisation
  - second order method since both gradient and Hessian are used
  - related Gauss-Newton algorithm employs Jacobian instead of Hessian

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno
  - Sequential Least Squares Programming
  - Levenberg–Marquardt
  - Migrad
  - Root Mean Square Propagation
- *Usage:* found in many modern libraries, suited to specialised use cases like nonlinear least squares

Application

Overview

Limitations & Curses

- *Convergence & Silent Failures:*
- *Dimensionality:*
- *Scaling:*
- *Convexity:*
- *Initialisation:*

Thank You!